Chapter 4: STL Algorithms Part 1

This is where STL becomes powerful.

Without STL:

```
// Sorting manually
// Binary Search manually
// Finding max manually
// Finding min manually
```

With STL:

```
sort()
binary_search()
max_element()
min_element()
```

One line.

Include Header

Most algorithms are in:

```
#include <algorithm>
```

For `accumulate()`:

```
#include <numeric>
```

---

1. sort()

Most important STL algorithm.

Syntax

```
sort(start, end);
```

Example:

```
vector<int> v = {5,2,9,1,7};

sort(v.begin(), v.end());
```

Output:

```
1 2 5 7 9
```

Complexity

```
O(n log n)
```

Array Sorting

```
int arr[] = {5,2,9,1};

sort(arr, arr+4);
```

Output:

```
1 2 5 9
```

Reverse Sorting

Method 1

```
sort(v.begin(), v.end(), greater<int>());
```

Example:

```
vector<int> v={5,2,9,1};

sort(v.begin(),v.end(),greater<int>());
```

Output:

```
9 5 2 1
```

Complexity

```
O(n log n)
```

Sorting Pair

Important Interview Question.

Example:

```
vector<pair<int,int>> v=
{
    {2,5},
    {1,3},
    {2,1},
    {1,1}
};

sort(v.begin(),v.end());
```

Output:

```
(1,1)
(1,3)
(2,1)
(2,5)
```

Rule:

1. Compare first
2. Then compare second

---

2. reverse()

Reverse container.

Syntax

```
reverse(start,end);
```

Example:

```
vector<int> v={1,2,3,4,5};

reverse(v.begin(),v.end());
```

Output:

```
5 4 3 2 1
```

Complexity

```
O(n)
```

Reverse Subarray

Example:

```
vector<int> v={1,2,3,4,5};

reverse(v.begin()+1,v.begin()+4);
```

Result:

```
1 4 3 2 5
```

---

3. find()

Search element.

Syntax

```
find(start,end,value)
```

Returns iterator.

Example:

```
vector<int> v={10,20,30,40};

auto it=find(v.begin(),v.end(),30);
```

Check found:

```
if(it!=v.end())
{
    cout<<"Found";
}
```

Output:

```
Found
```

Position of Element

```
cout<<it-v.begin();
```

Output:

```
2
```

Complexity

```
O(n)
```

---

4. count()

Count occurrences.

Syntax

```
count(start,end,value)
```

Example:

```
vector<int> v={1,2,2,2,3};

cout<<count(v.begin(),v.end(),2);
```

Output:

```
3
```

Complexity

```
O(n)
```

---

5. min_element()

Returns iterator to minimum.

Syntax

```
min_element(start,end)
```

Example:

```
vector<int> v={5,2,9,1,7};

auto it=min_element(v.begin(),v.end());

cout<<*it;
```

Output:

```
1
```

Position of Minimum

```
cout<<it-v.begin();
```

Output:

```
3
```

Complexity

```
O(n)
```

---

6. max_element()

Returns iterator to maximum.

Example:

```
vector<int> v={5,2,9,1,7};

auto it=max_element(v.begin(),v.end());

cout<<*it;
```

Output:

```
9
```

Complexity

```
O(n)
```

---

7. accumulate()

Sum of elements.

Header:

```
#include <numeric>
```

Syntax

```
accumulate(start,end,initial_value)
```

Example:

```
vector<int> v={1,2,3,4,5};

cout<<accumulate(v.begin(),v.end(),0);
```

Output:

```
15
```

Complexity

```
O(n)
```

Product of Elements

```
accumulate(v.begin(),v.end(),1,multiplies<int>());
```

Advanced.

---

8. binary_search()

Very Important.

Condition

Array must be sorted.

Syntax

```
binary_search(start,end,value)
```

Example:

```
vector<int> v={1,2,3,4,5};

cout<<binary_search(v.begin(),v.end(),4);
```

Output:

```
1
```

(True)

Example:

```
cout<<binary_search(v.begin(),v.end(),8);
```

Output:

```
0
```

(False)

Complexity

```
O(log n)
```

Useful Combination

Find Max

```
int mx=*max_element(v.begin(),v.end());
```

Find Min

```
int mn=*min_element(v.begin(),v.end());
```

Sum

```
int sum=accumulate(v.begin(),v.end(),0);
```

Sort Descending

```
sort(v.begin(),v.end(),greater<int>());
```

Check Existence

```
binary_search(v.begin(),v.end(),x);
```

Interview Example

Input:

```
5

4 2 7 1 9
```

Tasks:

1. Sort
2. Find max
3. Find min
4. Find sum

5. Search 7

Solution:

```
vector<int> v={4,2,7,1,9};

sort(v.begin(),v.end());

cout<<*min_element(v.begin(),v.end())<<endl;

cout<<*max_element(v.begin(),v.end())<<endl;

cout<<accumulate(v.begin(),v.end(),0)<<endl;

cout<<binary_search(v.begin(),v.end(),7);
```

Practice Questions

Easy

Q1

Sort array ascending.

Q2

Sort array descending.

Q3

Reverse vector.

Q4

Find largest element.

Q5

Find smallest element.

Q6

Find sum of all elements.

Q7

Count occurrences of 5.

Q8

Find if 10 exists.

Medium

Q9

Find second largest element using STL.

Hint:

```
sort()
```

Q10

Find difference between max and min.

Q11

Count distinct elements.

Hint:

```
sort()
```

Q12

Check if vector is palindrome.

Hint:

```
reverse()
```

Q13

Sort vector of pairs.

Input:

```
(2,5)
(1,3)
(2,1)
(1,1)
```

Interview Questions

Q14

Find frequency of every element.

(Hint: later using map)

Q15

Find kth largest element.

Hint:

```
sort descending
```

Q16

Find median.

Hint:

```
sort
```

Q17

Check if array contains duplicates.

Hint:

```
sort
```

```
count
```

Revision Sheet

Sorting

```
sort(begin,end);
```

Descending Sort

```
sort(begin,end,greater<int>());
```

Reverse

```
reverse(begin,end);
```

Search

```
find(begin,end,x);
```

Count

```
count(begin,end,x);
```

Minimum

```
min_element(begin,end);
```

Maximum

```
max_element(begin,end);
```

Sum

```
accumulate(begin,end,0);
```

Binary Search

```
binary_search(begin,end,x);
```

Requires sorted data.

STL Algorithms You Must Memorize

```
sort()
reverse()
find()
count()
min_element()
max_element()
accumulate()
binary_search()
```

These appear constantly in DSA rounds and interviews.

Assignment Before Next Chapter

Solve:

1. Sort ascending.
2. Sort descending.
3. Find max/min.
4. Sum of array.
5. Search element using binary_search().
6. Sort vector of pairs.
7. Find second largest.
8. Find kth largest.
9. Find median.

10. Check palindrome.
